

Day 1 — Monolith vs Microservices · Facilitator Guide

Companion to `labs/day-01-monolith-vs-microservices.md`. Facilitator-only — do not hand to participants.

Framing note

Today's job is to equip TPMs to frame the monolith-vs-microservices conversation in **business and operational terms** — not framework hype — and to write the architecture stance section of their TCD.

Activity 1 — Mono/Micro Triage

Cues:

- The "real-time dispatcher pricing engine" (card 6) is the trickiest: it serves many callers (suggests separation), but if it goes down, all dispatch fails (suggests resilience is a problem either way). Surface the trade-off.
- If a quad reaches for "microservices because Netflix" or similar — push back: Netflix's reasoning was about deploy independence at scale, not the technology itself.

Answer key / watch-for (triage pack stances):

- Card 1 (startup checkout, 50 customers, 1 team):
Monolith — no deploy-cadence contention, no scale pressure yet.
- Card 2 (streaming recommendations at scale, 3 teams):
Microservice — independent deploy cadence + distinct scale curve.
- Card 3 (new "saved searches" in existing monolith, one squad): **Modular monolith / module** — single owner, reversible.
- Card 4 (payments processor for 14 internal apps):
Microservice — independent failure domain + many

callers justify separation.

- Card 5 (nightly 3–10 min reporting export): **Need more info / async worker** — batch job; often a job/worker not a service split.
- Card 6 (real-time dispatcher pricing engine, all flows depend on it): the deliberate trap — serves many callers but is a single point of failure regardless. No clean answer; the point is naming the trade-off, not stamping a stance.
- Card 7 (small team's first internal tool, next to main app): **Monolith / module** — separation buys nothing here.
- Card 8 (load triples every World Cup, otherwise small): **Need more info** — bursty scale axis may justify separation *or* autoscaling within one runtime; press for evidence.

Activity 2 — Apply the Frame to Your PRD Feature

Cues:

- Framework hype ("microservices because Netflix") is the tell of an under-defended stance. Push to business terms.
- Two or three "we don't know" answers means the stance hasn't been earned — coach toward modular monolith as a starting point.

Activity 3 — Integration Map (first pass)

Cues:

- "Failure handling: TBD" is fine if it's an open question to discuss with the architect. "Failure handling: hopefully it works" is not.
- Push quads to think about **read-then-write** dependencies — they're the most common source of consistency bugs.

Activity 4 — AI-Assisted Architecture Q&A

Cues:

- Generic AI objections ("consider scalability") are the prompt's fault; coach toward Critique-hat prompts loaded with the quad's specific stance.

- Adopted-everything is the tell of unexamined AI use. Force at least one rejection with reasoning.

Facilitator reflection prompts (end of day)

- Which quad's stance was defended in framework hype rather than business terms? Coach tomorrow.
- Which quad's "Failure handling" column was the most concrete? Hold up Friday as a positive example.
- Did the AI-assisted exercises produce surprises, or just generic objections? If generic, the prompts need more context — tighten Week 4 Day 2.